FULL STACK WEB DEVELOPMENT LAB

Lab Sheet No.: 9

Course: B.Tech III Year – Full Stack Development Lab

Duration: 2 Hours

STUDENT DETAILS

Name: Priyanshu Vats

CU ID: CU24220142

Section: D

Roll No.: 45

EXPERIMENT TITLE

Design and Development of a "Student Course Registration System" using Frontend, Backend, and Database Integration

AIM

To design and implement a three-tier web application that allows a student to register for a course. The application uses frontend validation, a Flask backend API and MySQL database for storing registration details.

TECHNOLOGY USED

Frontend: HTML5, CSS3, JavaScript

Backend: Python Flask

Database: MySQL

API: REST API

Database Connector: mysql-connector-python

Editor: VS Code

PART A – DATABASE DESIGN

1. Create Database

```
CREATE DATABASE college_db;
USE college_db;
```

2. Create Courses Table

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    seats_available INT NOT NULL
);
```

3. Create Students Table

```
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    roll_no VARCHAR(30) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL,
    course_id INT,
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

4. Insert Sample Courses

```
INSERT INTO courses (course_name, seats_available)
VALUES
('Full Stack Web Development', 30),
('Artificial Intelligence', 25),
('Machine Learning', 20),
('Database Management System', 35);
```

5. Fetch Courses with Available Seats

```
SELECT *
FROM courses
WHERE seats_available > 0;
```

6. Insert a Student

```
INSERT INTO students (name, roll_no, email, course_id)
VALUES ('Priyanshu Vats', '31',
'priyanshu@example.com', 1);
```

7. Display Registered Students with Course Name

```
SELECT
    students.student_id,
    students.name,
    students.roll_no,
    students.email,
    courses.course_name
FROM students
JOIN courses
ON students.course_id = courses.course_id;
```

PART B – BACKEND API DEVELOPMENT

Project Structure

```
student-course-registration/
■
■■■ app.py
■■■ requirements.txt
■
■■■ templates/
■   ■■■ index.html
■
■■■ static/
    ■■■ style.css
    ■■■ script.js
```

Install Required Packages

```
pip install flask mysql-connector-python
```

Backend Code – app.py

```
from flask import Flask, request, jsonify, render_template
import mysql.connector

app = Flask(__name__)

def get_db_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="YOUR_PASSWORD",
        database="college_db"
    )

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/courses", methods=["GET"])
def get_courses():
    db = get_db_connection()
    cursor = db.cursor(dictionary=True)

    cursor.execute("""
        SELECT course_id, course_name, seats_available
        FROM courses
        WHERE seats_available > 0
    """)
    courses = cursor.fetchall()
    cursor.close()
    db.close()
    return jsonify(courses), 200

@app.route("/api/register", methods=["POST"])
def register_student():
    data = request.get_json()
    name = data.get("name")
    roll_no = data.get("roll_no")
    email = data.get("email")
    course_id = data.get("course_id")

    if not name or not roll_no or not email or not course_id:
        return jsonify({"message": "All fields are required"}), 400

    db = get_db_connection()
```

```
    cursor = db.cursor(dictionary=True)

    cursor.execute(
        "SELECT * FROM students WHERE roll_no = %s",
        (roll_no,)
    )
    existing_student = cursor.fetchone()

    if existing_student:
        cursor.close()
        db.close()
        return jsonify({"message": "Roll number is already registered"}), 409

    cursor.execute(
        "SELECT seats_available FROM courses WHERE course_id = %s",
        (course_id,)
    )
    course = cursor.fetchone()

    if not course:
        cursor.close()
        db.close()
        return jsonify({"message": "Course not found"}), 400

    if course["seats_available"] <= 0:
        cursor.close()
        db.close()
        return jsonify({"message": "No seats available for this course"}), 409

    cursor.execute("""
        INSERT INTO students
        (name, roll_no, email, course_id)
        VALUES (%s, %s, %s, %s)
    """, (name, roll_no, email, course_id))

    cursor.execute("""
        UPDATE courses
        SET seats_available = seats_available - 1
        WHERE course_id = %s
    """, (course_id,))

    db.commit()
    cursor.close()
    db.close()

    return jsonify({"message": "Student registered successfully"}), 201

@app.route("/api/students", methods=["GET"])
def get_students():
    db = get_db_connection()
    cursor = db.cursor(dictionary=True)

    cursor.execute("""
        SELECT
            students.student_id,
            students.name,
            students.roll_no,
            students.email,
            courses.course_name
        FROM students
        JOIN courses
        ON students.course_id = courses.course_id
    """)
    students = cursor.fetchall()
    cursor.close()
    db.close()
    return jsonify(students), 200

if __name__== "__main__":
    app.run(debug=True)
```

REST API ENDPOINTS

Method	Endpoint	Purpose
GET	/api/courses	Displays available courses
POST	/api/register	Registers a student
GET	/api/students	Displays registered students

The backend checks for empty values, duplicate roll numbers and course seat availability before inserting the registration.

PART C – FRONTEND DEVELOPMENT

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Course Registration</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>
    <div class="container">
        <h1>Student Course Registration System</h1>
        <form id="registrationForm">
            <label>Student Name</label>
            <input type="text" id="name" placeholder="Enter your name" required>
            <label>Roll No.</label>
            <input type="text" id="roll_no" placeholder="Enter roll number" required>
            <label>Email</label>
            <input type="email" id="email" placeholder="Enter email" required>
            <label>Select Course</label>
            <select id="course_id" required>
                <option value="">Select Course</option>
            </select>
            <button type="submit">Register</button>
        </form>
        <p id="message"></p>
        <h2>Registered Students</h2>
        <table>
            <thead>
                <tr>
                    <th>Name</th><th>Roll No.</th><th>Email</th><th>Course</th>
                </tr>
            </thead>
            <tbody id="studentTable"></tbody>
        </table>
    </div>
    <script src="/static/script.js"></script>
</body>
</html>
```

CSS – style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 30px;
}
.container {
    max-width: 900px;
    margin: auto;
    background: white;
    padding: 25px;
    border-radius: 10px;
}
h1 { text-align: center; }
form {
    display: flex;
    flex-direction: column;
    gap: 10px;
}
input, select {
    padding: 10px;
    border: 1px solid #aaa;
    border-radius: 5px;
}
button {
    padding: 12px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    background-color: #333;
    color: white;
}
table {
    width: 100%;
    margin-top: 20px;
    border-collapse: collapse;
}
th, td {
    border: 1px solid #ccc;
    padding: 10px;
    text-align: center;
}
th { background-color: #eee; }
```

JavaScript – script.js

```
const courseSelect = document.getElementById("course_id");
const form = document.getElementById("registrationForm");
const message = document.getElementById("message");
```

```
const studentTable = document.getElementById("studentTable");

async function loadCourses() {
    const response = await fetch("/api/courses");
    const courses = await response.json();

    courseSelect.innerHTML =
        '<option value="">Select Course</option>';

    courses.forEach(course => {
        const option = document.createElement("option");
        option.value = course.course_id;
        option.textContent =
            `${course.course_name} (${course.seats_available} seats)`;
        courseSelect.appendChild(option);
    });
}

form.addEventListener("submit", async function(event) {
    event.preventDefault();

    const name = document.getElementById("name").value.trim();
    const roll_no = document.getElementById("roll_no").value.trim();
    const email = document.getElementById("email").value.trim();
    const course_id = courseSelect.value;

    if (!name || !roll_no || !email || !course_id) {
        message.textContent = "Please fill all fields.";
        return;
    }

    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (!emailPattern.test(email)) {
        message.textContent = "Please enter a valid email.";
        return;
    }

    const response = await fetch("/api/register", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({name, roll_no, email, course_id})
    });

    const result = await response.json();
    message.textContent = result.message;

    if (response.ok) {
        form.reset();
        loadCourses();
        loadStudents();
    }
});

async function loadStudents() {
    const response = await fetch("/api/students");
    const students = await response.json();
    studentTable.innerHTML = "";

    students.forEach(student => {
        const row = document.createElement("tr");
        row.innerHTML = `
            <td>${student.name}</td>
            <td>${student.roll_no}</td>
            <td>${student.email}</td>
            <td>${student.course_name}</td>
        `;
        studentTable.appendChild(row);
    });
}

loadCourses();
loadStudents();
```

PART D – INTEGRATION AND TESTING

Step 1 – Start MySQL

```
USE college_db;
SELECT * FROM courses;
SELECT * FROM students;
```

Step 2 – Run Flask Application

```
python app.py

Open in browser:
http://127.0.0.1:5000
```

Step 3 – Test Registration

```
Name: Priyanshu Vats
Roll No.: 31
Email: priyanshu@example.com
Course: Full Stack Web Development
```

Step 4 – Verify Database

```
SELECT * FROM students;
```

Step 5 – Test Duplicate Roll Number

```
Roll No.: 31

Backend response:
Roll number is already registered

HTTP status:
409 Conflict
```

Step 6 – Test No Seat Available

```
If a course has seats_available = 0, the backend does not allow a new registration and returns:
No seats available for this course
```

WORKING OF THE APPLICATION

```
Student
   ↓
Frontend Form
   ↓
JavaScript Validation
   ↓
REST API
   ↓
Flask Backend
   ↓
MySQL Database
   ↓
Flask API Response
   ↓
Frontend
```

The frontend sends registration information to the Flask backend using a POST request. The backend validates the information and checks the database. If the registration is valid, the student record is inserted and the available seats are decreased.

EXPECTED OUTCOME

A three-tier Student Course Registration System is successfully developed.

1. Frontend to backend communication using REST API.
2. Client-side form validation.
3. Backend validation.
4. Database connectivity using MySQL.
5. Course and student tables.
6. Foreign key relationship.
7. Dynamic course loading.
8. Student registration.
9. Duplicate roll number checking.
10. Seat availability checking.
11. Display of registered students.

Complete data flow: UI → API → Database → API → UI

SAMPLE VIVA QUESTIONS AND ANSWERS

1. Why is client-side validation not sufficient?

Client-side validation improves user experience, but it can be bypassed by the user. Therefore, backend validation is also required to make sure that invalid or malicious data is not stored in the database.

2. What is the role of a foreign key?

A foreign key connects the students table with the courses table. The course_id in the students table refers to the course_id of the courses table. It maintains the relationship between students and their selected courses.

3. Difference between GET and POST

GET is generally used to retrieve data from the server. Example: GET /api/courses. POST is used to send data to the server for creating a new record. Example: POST /api/register.

4. What HTTP status code is used when seats are not available?

I used 409 Conflict because the registration request conflicts with the current availability of the course.

5. How can SQL injection be prevented?

SQL injection can be prevented by using parameterized queries instead of directly joining user input with SQL statements.

```
cursor.execute(
    "SELECT * FROM students WHERE roll_no = %s",
    (roll_no,)
)
```

This makes the query safer because the user input is passed separately as a parameter.

CONCLUSION

I successfully designed and developed a Student Course Registration System using HTML, CSS, JavaScript, Python Flask and MySQL.

The project helped me understand how the frontend, backend API and database work together in a full-stack web application. I also implemented validation for duplicate roll numbers and course seat availability.

Name: Priyanshu Vats

CU ID: CU24220142

Section: D

Roll No.: 45

FULL STACK DEVELOPMENT · LAB SHEET 10

CampusConnect – Role-Based Event and Announcement Portal

Name: Priyanshu Vats

Course: B.Tech III Year – Full Stack Development Lab

Experiment: Secure Full-Stack CampusConnect Application

Problem Statement: Build a role-based college event and announcement portal. ADMIN can create, edit and delete events/announcements. STUDENT can view events, RSVP and receive live notifications. The application uses JWT authentication, Socket.io, Redis, React, MongoDB and Docker.

Technologies Used: Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Socket.io, Redis, React, Redux Toolkit, Axios, Jest, Supertest, React Testing Library, Docker and Docker Compose.

TASK 1 — AUTHENTICATION & RBAC

Aim: To create secure registration/login APIs and restrict routes according to ADMIN and STUDENT roles.

1. User Schema (models/User.js)

```
const mongoose = require("mongoose");
const bcrypt = require("bcrypt");

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  role: {
    type: String,
    enum: ["ADMIN", "STUDENT"],
    default: "STUDENT"
  }
});

userSchema.pre("save", async function(next) {
  if (!this.isModified("passwordHash")) return next();
  this.passwordHash = await bcrypt.hash(this.passwordHash, 10);
  next();
});

module.exports = mongoose.model("User", userSchema);
```

2. Authentication Routes (routes/auth.js)

```
const express = require("express");
const jwt = require("jsonwebtoken");
const bcrypt = require("bcrypt");
const User = require("../models/User");

const router = express.Router();

router.post("/register", async (req, res) => {
  try {
    const { name, email, password, role } = req.body;

    const oldUser = await User.findOne({ email });
    if (oldUser) return res.status(409).json({ message: "Email already registered" });

    const user = await User.create({
      name,
      email,
      passwordHash: password,
      role: role === "ADMIN" ? "ADMIN" : "STUDENT"
    });

    res.status(201).json({
      message: "Registration successful",
      user: { id: user._id, name: user.name, role: user.role }
    });
  } catch (err) {
    res.status(500).json({ message: "Server error" });
  }
});

router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });

    if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
      return res.status(401).json({ message: "Invalid email or password" });
    }

    const accessToken = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: "15m" }
    );

    const refreshToken = jwt.sign(
      { id: user._id },
      process.env.REFRESH_SECRET,
      { expiresIn: "7d" }
    );

    res.cookie("refreshToken", refreshToken, {
      httpOnly: true,
      sameSite: "strict",
      secure: process.env.NODE_ENV === "production",
      maxAge: 7 * 24 * 60 * 60 * 1000
```

```
    });

    res.json({
      accessToken,
      user: { id: user._id, name: user.name, role: user.role }
    });
  } catch (err) {
    res.status(500).json({ message: "Server error" });
  }
});

router.post("/refresh", (req, res) => {
  try {
    const token = req.cookies.refreshToken;
    if (!token) return res.status(401).json({ message: "No refresh token" });

    const data = jwt.verify(token, process.env.REFRESH_SECRET);
    const accessToken = jwt.sign(
      { id: data.id },
      process.env.JWT_SECRET,
      { expiresIn: "15m" }
    );

    res.json({ accessToken });
  } catch (err) {
    res.status(401).json({ message: "Invalid refresh token" });
  }
});

module.exports = router;
```

3. Authentication and Authorization Middleware (middleware/auth.js)

```
const jwt = require("jsonwebtoken");

function authenticate(req, res, next) {
  const header = req.headers.authorization;

  if (!header || !header.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Access token required" });
  }

  const token = header.split(" ")[1];

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}

function authorize(role) {
  return (req, res, next) => {
    if (!req.user || req.user.role !== role) {
      return res.status(403).json({ message: "Access denied" });
    }
    next();
  };
}

module.exports = { authenticate, authorize };
```

4. Example Admin-only Route

```
router.post("/api/events",
  authenticate,
  authorize("ADMIN"),
  async (req, res) => {
    const event = await Event.create(req.body);
    res.status(201).json(event);
  }
);
```

Expected result: A logged-in ADMIN can create an event. A STUDENT trying the same endpoint receives HTTP 403.

TASK 2 — REAL-TIME NOTIFICATIONS

Aim: To send an instant notification to connected students whenever an admin posts an announcement.

1. Socket.io Server Setup

```
const { Server } = require("socket.io");
```

```
const jwt = require("jsonwebtoken");

const io = new Server(server, {
  cors: {
    origin: process.env.FRONTEND_URL,
    credentials: true
  }
});

io.use((socket, next) => {
  try {
    const token = socket.handshake.auth.token;
    const user = jwt.verify(token, process.env.JWT_SECRET);
    socket.user = user;
    next();
  } catch (err) {
    next(new Error("Socket authentication failed"));
  }
});

io.on("connection", (socket) => {
  console.log("Socket connected:", socket.id);

  if (socket.user.role === "STUDENT") {
    socket.join("students");
  }

  socket.on("disconnect", () => {
    console.log("Socket disconnected:", socket.id);
  });
});
```

2. Emit Notification After Announcement Creation

```
router.post("/api/announcements",
  authenticate,
  authorize("ADMIN"),
  async (req, res) => {
    const announcement = await Announcement.create(req.body);

    io.to("students").emit("new-announcement", {
      id: announcement._id,
      title: announcement.title,
      message: announcement.message
    });

    res.status(201).json(announcement);
  }
);
```

3. React Client Socket Code

```
import { useEffect, useState } from "react";
import { io } from "socket.io-client";

function NotificationBox({ token }) {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const socket = io("http://localhost:5000", {
      auth: { token },
      reconnection: true,
      reconnectionAttempts: 5
    });

    socket.on("new-announcement", (data) => {
      setCount((old) => old + 1);
      alert("New Announcement: " + data.title);
    });

    return () => socket.disconnect();
  }, [token]);

  return <div>New notifications: {count}</div>;
}

export default NotificationBox;
```

TASK 3 — REDIS CACHING LAYER

Aim: To cache the event list for 60 seconds so repeated requests do not always query MongoDB.

1. Redis Client (config/redis.js)

```
const { createClient } = require("redis");
```

```
const redisClient = createClient({
  url: process.env.REDIS_URL || "redis://redis:6379"
});

redisClient.on("error", (err) => console.log("Redis Error", err));

async function connectRedis() {
  await redisClient.connect();
  console.log("Redis connected");
}

module.exports = { redisClient, connectRedis };
```

2. GET /api/events with 60-second Cache

```
router.get("/api/events", async (req, res) => {
  const cacheKey = "events:list";

  const cached = await redisClient.get(cacheKey);
  if (cached) {
    return res.json({
      source: "cache",
      data: JSON.parse(cached)
    });
  }

  const events = await Event.find().sort({ date: 1 });

  await redisClient.setEx(
    cacheKey,
    60,
    JSON.stringify(events)
  );

  res.json({
    source: "database",
    data: events
  });
});
```

3. Cache Invalidation

```
async function clearEventCache() {
  await redisClient.del("events:list");
}

router.post("/api/events",
  authenticate,
  authorize("ADMIN"),
  async (req, res) => {
    const event = await Event.create(req.body);
    await clearEventCache();
    res.status(201).json(event);
  }
);

router.put("/api/events/:id",
  authenticate,
  authorize("ADMIN"),
  async (req, res) => {
    const event = await Event.findByIdAndUpdate(
      req.params.id, req.body, { new: true }
    );
    await clearEventCache();
    res.json(event);
  }
);

router.delete("/api/events/:id",
  authenticate,
  authorize("ADMIN"),
  async (req, res) => {
    await Event.findByIdAndDelete(req.params.id);
    await clearEventCache();
    res.json({ message: "Event deleted" });
  }
);
```

4. Benchmark using Autocannon

```
npm install -D autocannon

# Uncached / normal request test
npx autocannon -c 10 -d 10 http://localhost:5000/api/events

# Run the same command after the first request has populated Redis
npx autocannon -c 10 -d 10 http://localhost:5000/api/events
```

Observation table to fill after running locally:

Request type	No. of requests	Average response time	Source
Without cache	100+	_____ms	MongoDB
With Redis cache	100+	_____ms	Redis

Expected observation: After the first database request, repeated GET /api/events requests are served from Redis until the 60-second TTL expires. Exact response times depend on the computer and Docker setup, so they should be recorded from the actual benchmark instead of using a made-up value.

TASK 4 — FRONTEND WITH STATE MANAGEMENT

Aim: To create a role-aware React frontend and manage authentication/events globally using Redux Toolkit.

1. Redux Store

```
// store.js
import { configureStore } from "@reduxjs/toolkit";
import authReducer from "./authSlice";
import eventReducer from "./eventSlice";

export const store = configureStore({
  reducer: {
    auth: authReducer,
    events: eventReducer
  }
});
```

2. Authentication Slice

```
// authSlice.js
import { createSlice } from "@reduxjs/toolkit";

const authSlice = createSlice({
  name: "auth",
  initialState: {
    user: null,
    accessToken: null
  },
  reducers: {
    loginSuccess: (state, action) => {
      state.user = action.payload.user;
      state.accessToken = action.payload.accessToken;
    },
    logout: (state) => {
      state.user = null;
      state.accessToken = null;
    }
  }
});

export const { loginSuccess, logout } = authSlice.actions;
export default authSlice.reducer;
```

3. Event Slice

```
// eventSlice.js
import { createSlice } from "@reduxjs/toolkit";

const eventSlice = createSlice({
  name: "events",
  initialState: { list: [], loading: false },
  reducers: {
    setEvents: (state, action) => {
      state.list = action.payload;
    },
    setLoading: (state, action) => {
      state.loading = action.payload;
    }
  }
});

export const { setEvents, setLoading } = eventSlice.actions;
export default eventSlice.reducer;
```

4. Axios Interceptor

```
import axios from "axios";
```

```
import { store } from "./store";

const api = axios.create({
  baseURL: "http://localhost:5000/api",
  withCredentials: true
});

api.interceptors.request.use((config) => {
  const token = store.getState().auth.accessToken;

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});

export default api;
```

5. Login Component

```
import { useState } from "react";
import { useDispatch } from "react-redux";
import api from "./api";
import { loginSuccess } from "./authSlice";

function Login() {
  const dispatch = useDispatch();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const submit = async (e) => {
    e.preventDefault();

    try {
      const { data } = await api.post("/auth/login", {
        email,
        password
      });

      dispatch(loginSuccess(data));
    } catch (err) {
      alert(err.response?.data?.message || "Login failed");
    }
  };

  return (
    <form onSubmit={submit}>
      <h2>CampusConnect Login</h2>
      <input
        placeholder="Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  );
}

export default Login;
```

6. Event List with Search

```
import { useEffect, useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import api from "./api";
import { setEvents } from "./eventSlice";

function EventList() {
  const dispatch = useDispatch();
  const events = useSelector((s) => s.events.list);
  const [search, setSearch] = useState("");

  useEffect(() => {
    api.get("/events").then(({ data }) => {
      dispatch(setEvents(data.data));
    });
  }, [dispatch]);

  const filtered = events.filter((event) =>
    event.title.toLowerCase().includes(search.toLowerCase())
```

```
  );

  return (
    <div>
      <h2>College Events</h2>
      <input
        placeholder="Search by title"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      {filtered.map((event) => (
        <div key={event._id}>
          <h3>{event.title}</h3>
          <p>{event.description}</p>
          <button>RSVP</button>
        </div>
      ))}
    </div>
  );
}

export default EventList;
```

7. PrivateRoute / RoleRoute

```
import { Navigate } from "react-router-dom";
import { useSelector } from "react-redux";

function PrivateRoute({ children, role }) {
  const { user, accessToken } = useSelector((s) => s.auth);

  if (!accessToken) return <Navigate to="/login" />;

  if (role && user?.role !== role) {
    return <Navigate to="/dashboard" />;
  }

  return children;
}

export default PrivateRoute;
```

TASK 5 — SECURITY HARDENING

Aim: To add rate limiting, request validation, restricted CORS and security headers.

1. Install Security Packages

```
npm install express-rate-limit zod helmet cors cookie-parser
```

2. Express Security Configuration

```
const helmet = require("helmet");
const cors = require("cors");
const rateLimit = require("express-rate-limit");

app.use(helmet());

app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true
}));

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  standardHeaders: true,
  legacyHeaders: false,
  message: { message: "Too many login attempts. Try again later." }
});

app.use("/api/auth/login",  loginLimiter);
```

3. Zod Validation

```
const { z } = require("zod");

const registerSchema = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  password: z.string().min(6)
});

function validate(schema) {
```

```
  return (req, res, next) => {
    const result = schema.safeParse(req.body);

    if (!result.success) {
      return res.status(400).json({
        message: "Validation error",
        errors: result.error.issues
      });
    }

    req.body = result.data;
    next();
  };
}

router.post(
  "/register",
  validate(registerSchema),
  registerController
);
```

Security points used: passwords are hashed with bcrypt; JWT is checked by middleware; refresh token is stored in an httpOnly cookie; login attempts are limited; request data is validated; CORS is restricted; Helmet adds security headers.

TASK 6 — TESTING

Aim: To test the important backend APIs and React components.

1. Backend Tests (Jest + Supertest)

```
const request = require("supertest");
const app = require("../app");

describe("CampusConnect API", () => {

  test("registration should work", async () => {
    const res = await request(app)
      .post("/api/auth/register")
      .send({
        name: "Test Student",
        email: "teststudent@example.com",
        password: "pass1234"
      });

    expect([201, 409]).toContain(res.statusCode);
  });

  test("wrong password should fail login", async () => {
    const res = await request(app)
      .post("/api/auth/login")
      .send({
        email: "teststudent@example.com",
        password: "wrongpassword"
      });

    expect(res.statusCode).toBe(401);
  });

  test("protected route rejects request without token", async () => {
    const res = await request(app)
      .get("/api/profile");

    expect(res.statusCode).toBe(401);
  });

  test("student cannot access admin-only route", async () => {
    const studentToken = "VALID_STUDENT_TOKEN_FOR_TEST";

    const res = await request(app)
      .post("/api/events")
      .set("Authorization", `Bearer ${studentToken}`)
      .send({
        title: "Test Event"
      });

    expect(res.statusCode).toBe(403);
  });

  test("admin can create event", async () => {
    const adminToken = "VALID_ADMIN_TOKEN_FOR_TEST";

    const res = await request(app)
      .post("/api/events")
```

```
      .set("Authorization", `Bearer ${adminToken}`)
      .send({
        title: "Coding Workshop",
        description: "Workshop for students",
        date: "2026-10-15"
      });

    expect(res.statusCode).toBe(201);
  });
});
```

Note: In the actual project, the placeholder tokens above are replaced by tokens generated during the test setup or by a test database fixture.

2. React Testing Library Tests

```
import { render, screen } from "@testing-library/react";
import { Provider } from "react-redux";
import { store } from "../store";
import Login from "../Login";
import EventList from "../EventList";

test("Login form is displayed", () => {
  render(
    <Provider store={store}>
      <Login />
    </Provider>
  );

  expect(screen.getByText("CampusConnect Login")).toBeInTheDocument();
  expect(screen.getByPlaceholderText("Email")).toBeInTheDocument();
  expect(screen.getByPlaceholderText("Password")).toBeInTheDocument();
});

test("Login button is visible", () => {
  render(
    <Provider store={store}>
      <Login />
    </Provider>
  );

  expect(screen.getByRole("button", { name: /login/i }))
    .toBeInTheDocument();
});

test("Event List heading is displayed", () => {
  render(
    <Provider store={store}>
      <EventList />
    </Provider>
  );

  expect(screen.getByText("College  Events")).toBeInTheDocument();
});
```

3. Run Tests

```
# Backend
npm test

# React tests
npm test ------watchAll=false
```

TASK 7 — CONTAINERIZATION & DEPLOYMENT

Aim: To run the complete CampusConnect stack using Docker Compose with separate frontend, backend, MongoDB and Redis containers.

1. Backend Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

2. Frontend Dockerfile (multi-stage build)

```
FROM node:20-alpine AS build

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

3. docker-compose.yml

```
services:

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      PORT: 5000
      MONGO_URI: mongodb://mongo:27017/campusconnect
      REDIS_URL: redis://redis:6379
      JWT_SECRET: campusconnect_access_secret
      REFRESH_SECRET: campusconnect_refresh_secret
      FRONTEND_URL: http://localhost
      NODE_ENV: development
    depends_on:
      - mongo
      - redis

  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend

  mongo:
    image: mongo:7
    volumes:
      - mongo_data:/data/db
    ports:
      - "27017:27017"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  mongo_data:
```

4. Suggested Project Structure

```
CampusConnect/
■
■■■ backend/
■   ■■■ models/
■   ■■■ routes/
■   ■■■ middleware/
■   ■■■ config/
■   ■■■ tests/
■   ■■■ app.js
■   ■■■ server.js
■   ■■■ package.json
■   ■■■ Dockerfile
■
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ store.js
■   ■   ■■■ authSlice.js
■   ■   ■■■ eventSlice.js
■   ■   ■■■ api.js
■   ■■■ package.json
■   ■■■ Dockerfile
■   ■■■ nginx.conf
■
■■■ docker-compose.yml
```

5. Start the Complete Application

```
# From the CampusConnect folder

docker compose up --build
```

Verification: After startup, the frontend container serves the React application, the backend communicates with MongoDB and Redis through the Compose network, and MongoDB keeps its data in the mongo_data volume. The stack can be stopped with Ctrl+C or with docker compose down.

Lab Work Checklist

■ Task 1: JWT authentication, refresh token and ADMIN/STUDENT RBAC implemented.

■ Task 2: Socket.io live announcement notification implemented.

■ Task 3: Redis event caching with 60-second TTL and cache invalidation implemented.

■ Task 4: React + Redux Toolkit, Axios interceptor, search and protected routes included.

■ Task 5: bcrypt, rate limiting, Zod validation, CORS and Helmet included.

■ Task 6: Backend and React testing examples included.

■ Task 7: Backend/frontend Dockerfiles and docker-compose.yml included.